浙江理工大学科技与艺术学院

KEYI COLLEGE OF ZHEJIANG SCI-TECH UNIVERSITY

程序设计基础

大作业报告

大作业名称 ：科艺校内官网新闻资讯抓取及本地化存储

大作业时间：2025/12/20

专　　业　大数据管理与应用

班　　级　大数据管理与应用1

学生姓名　杨济同

学　　号　Xc24590104

指导老师　叶杰锋

一、需求分析

本次大作业的整体任务是开发一款定向爬虫工具，实现对浙江理工大学科技与艺术学院官网核心资讯栏目的自动化数据抓取与结构化存储，解决人工浏览、整理资讯效率低、数据分散的问题，为后续资讯检索、发布趋势分析等场景提供基础数据支持。目标站点为浙江理工大学科技与艺术学院官网的两个核心资讯栏目，具体 URL 如下：科艺要闻栏目：https://www.ky.zstu.edu.cn/kyyw.htm（发布学院重要会议、政策、活动等核心资讯）；校园经纬栏目：https://www.ky.zstu.edu.cn/xyjw.htm（发布校园生活、师生动态、校园建设等日常资讯）最终目标实现“自动请求→数据解析→去重存储”全流程自动化，支持单次运行同时抓取两个栏目数据；

二、设计思路

Python 3.8+（兼容性强，爬虫生态完善），PyCharm 2023.2，库：requests：发送 HTTP 请求，获取网页 HTML 源码；lxml（etree）：解析 HTML 结构，提取目标数据；csv：处理本地 CSV 文件写入；pymongo：连接 MongoDB 数据库，实现数据存储；time：设置防反爬延迟，避免触发官网反爬机制；数据库：MongoDB 5.0+（非关系型数据库，适合存储非结构化 / 半结构化的资讯数据，支持灵活查询）。

核心设计思路采用 “通用化模块拆分” 设计，将爬虫流程拆分为 “配置层→抓取层→存储层”，降低代码耦合度，便于后续扩展（如新增其他资讯栏目）；配置层：集中管理所有可变参数（如目标 URL、保存路径、数据库信息），修改时无需改动核心逻辑；抓取层：设计通用抓取函数 fetch_news_data，适配两个栏目相同的页面结构，避免重复编写解析代码；存储层：拆分 save_to_csv 和 save_to_mongo 两个通用存储函数，支持数据多端同步存储，满足不同使用场景。



图 2-1（流程图）

三、详细设计

任务 1：

1.网络请求的 Network 抓取并分析

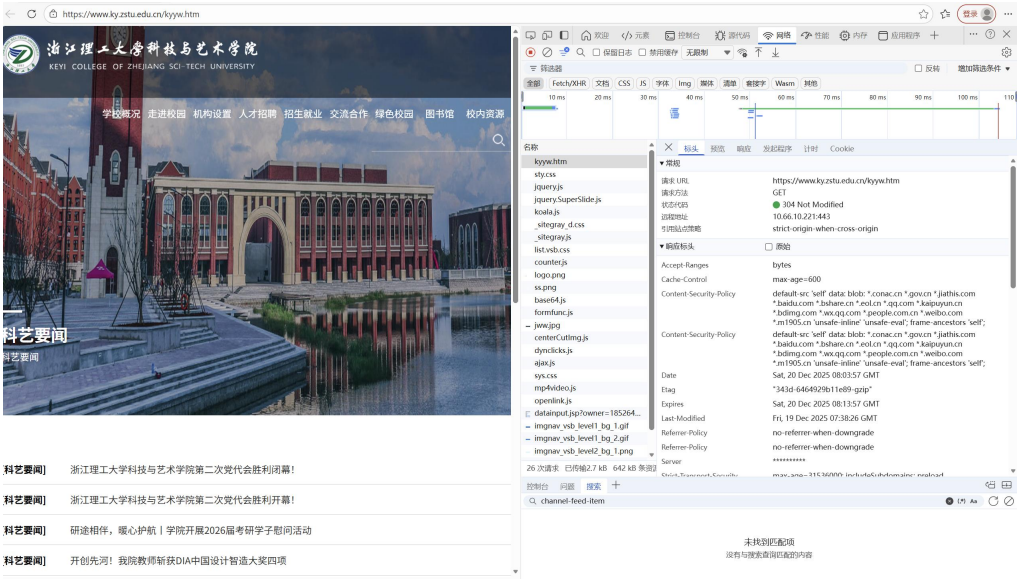


图 3-1（科艺要闻页面）

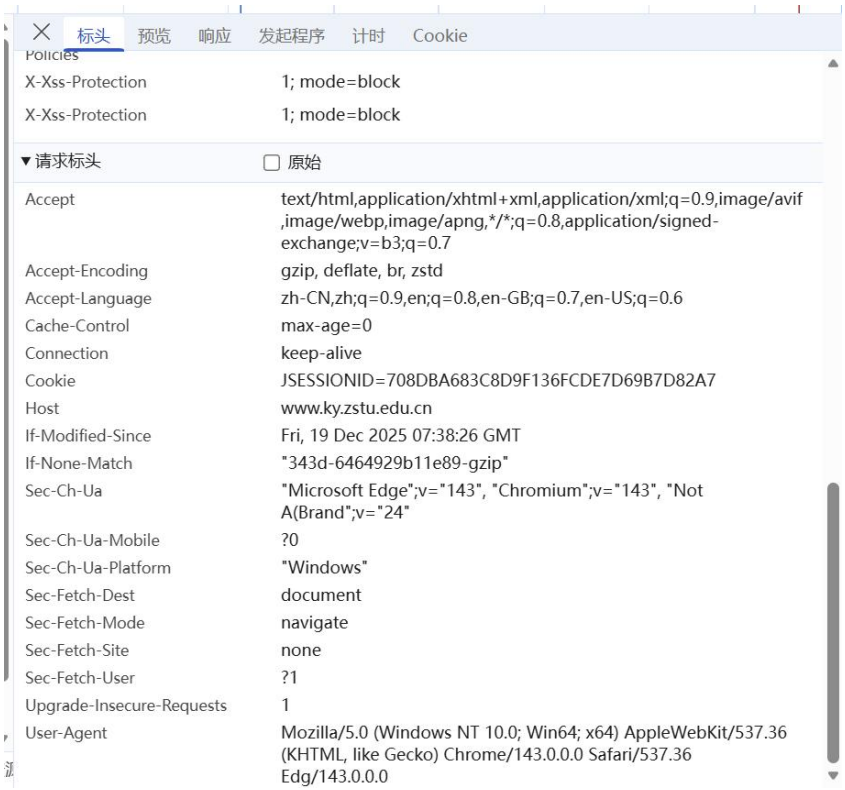


图 3-2（科艺要闻页面）

打开通知网站，打开浏览器开发者工具，打开后的如图所示，确定该网站的请求方式为 GET，确定 URL，Cookie，Host，Referer，User-Agent 等信息。

2.页面结构的 HTML 分析

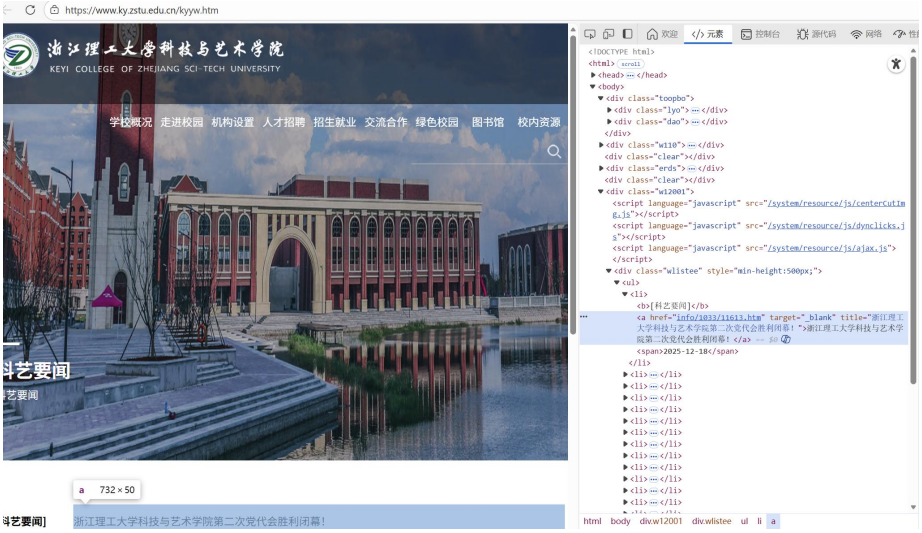


图 3-3（HTML 结构）

3.代码编写



图 3-4（python 库）



图 3-5（基础配置）

进行基础配置（数据库存储使用 MongoDB；配置请求头和对应的 URL）

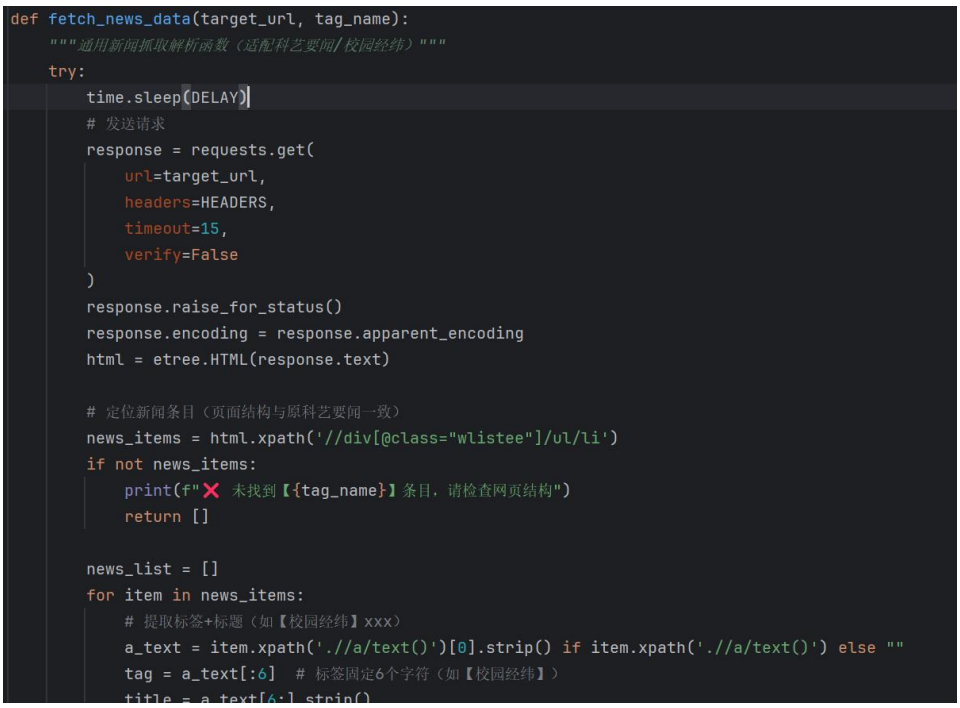
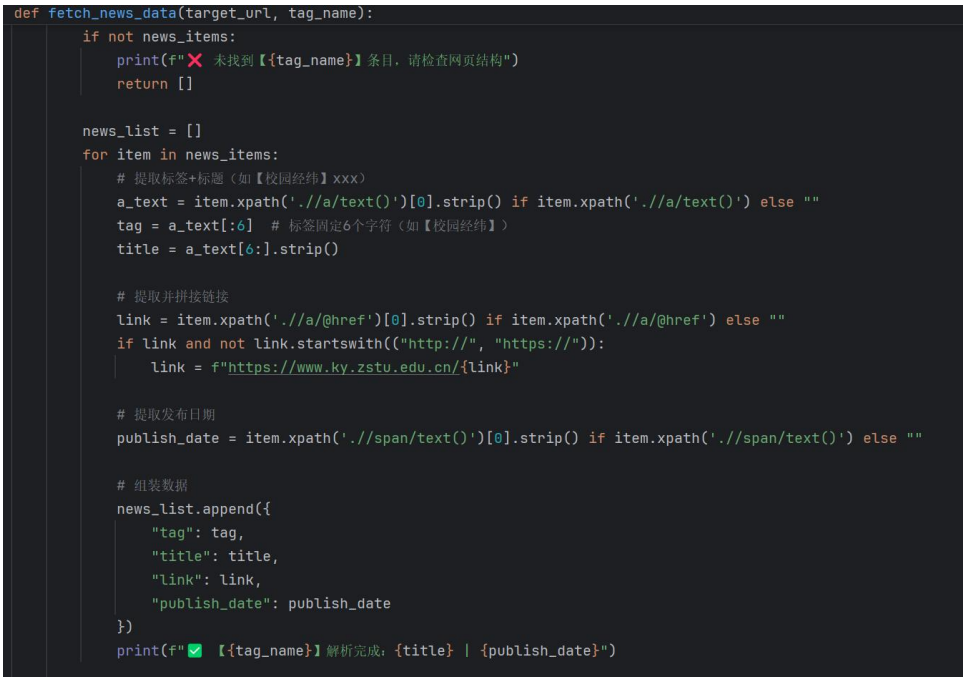


图 3-6（定义函数）

函数开头的 try 块首先执行 time.sleep(DELAY)，这一步是为了实现防反爬延迟，

避免短时间内频繁请求目标网站导致 IP 被封禁，让请求节奏更贴近人工浏览；接着通过 requests.get 发送 HTTP GET 请求，其中 url 对应传入的目标栏目地址，headers=HEADERS 带上了模拟浏览器的请求头（避免被网站识别为非浏览器爬虫），timeout=15 设置 15 秒超时防止请求阻塞，verify=False 则跳过 SSL 证书验证，解决部分网站证书异常导致的请求失败问题。



```
def fetch_news_data(target_url, tag_name):
    if not news_items:
        print(f"❌ 未找到【{tag_name}】条目，请检查网页结构")
        return []

    news_list = []
    for item in news_items:
        # 提取标签+标题（如【校园经纬】xxx）
        a_text = item.xpath('.//a/text()')[0].strip() if item.xpath('.//a/text()') else ""
        tag = a_text[:6]  # 标签固定6个字符（如【校园经纬】）
        title = a_text[6:].strip()

        # 提取并拼接链接
        link = item.xpath('.//a/@href')[0].strip() if item.xpath('.//a/@href') else ""
        if link and not link.startswith(("http://", "https://")):
            link = f"https://www.ky.zstu.edu.cn/{link}"

        # 提取发布日期
        publish_date = item.xpath('.//span/text()')[0].strip() if item.xpath('.//span/text()') else ""

        # 组装数据
        news_list.append({
            "tag": tag,
            "title": title,
            "link": link,
            "publish_date": publish_date
        })
        print(f"✅ 【{tag_name}】解析完成：{title} | {publish_date}")
```

图 3-7（定义函数）

请求发送后，response.raise_for_status()会主动捕获 HTTP 错误（如 404 页面不存在、403 权限拒绝等），若请求状态码异常则直接抛出错误；response.encoding = response.apparent_encoding 是为了处理网页编码乱码 ——apparent_encoding 会自动识别页面真实编码，替代默认从响应头获取的编码，确保后续解析的文本无乱码；随后用 etree.HTML(response.text)将网页源码转为 lxml 可解析的 HTML 对象，方便后续用 XPath 提取数据。之后通过 html.xpath('//div[@class="wlistee"]/ul/li')定位新闻条目，这个 XPath 是根据目标网站的页面结构确定的（新闻内容嵌套在 class 为“wlistee” 的 div 下的 ul 列表 li 标签中）；若该 XPath 未匹配到任何内容，会打印 “未找到条目” 的提示（说明网页结构可能变更），并返回空列表结束这部分逻辑。

```
# 组装数据
news_list.append({
    "tag": tag,
    "title": title,
    "link": link,
    "publish_date": publish_date
})
print(f"✅ 【{tag_name}】解析完成：{title} | {publish_date}")
```

图 3-8（结构化数据）

进入新闻条目遍历环节后，先初始化 news_list 空列表用于存储结构化数据。遍历每个新闻条目 item 时，首先通过 item.xpath('.//a/text()')提取 a 标签文本（格式为“【栏目名】新闻标题”），因此用 a_text[:6]截取前 6 个字符作为新闻标签（如“【校园经纬】”），再用 a_text[6:].strip()截取剩余部分作为新闻标题并去除空白字符；之后提取链接时，通过 item.xpath('.//a/@href')获取 a 标签的 href 属性，若链接是相对路径（未以 “http://”“https://” 开头），则拼接目标网站根域名转为绝对路径，避免后续无法直接访问详情页；再通过 item.xpath('.//span/text()')提取发布日期，同样做了空值兼容处理。最后将标签、标题、链接、发布日期组装成字典，添加到 news_list 中，同时打印解析完成的提示（展示标题和日期），方便调试查看进度。

```
    return news_list

except requests.exceptions.RequestException as e:
    print(f"❌ 【{tag_name}】请求失败：{str(e)}")
    return []
except Exception as e:
    print(f"❌ 【{tag_name}】解析失败：{str(e)}")
    return []
```

图 3-9（响应处理）

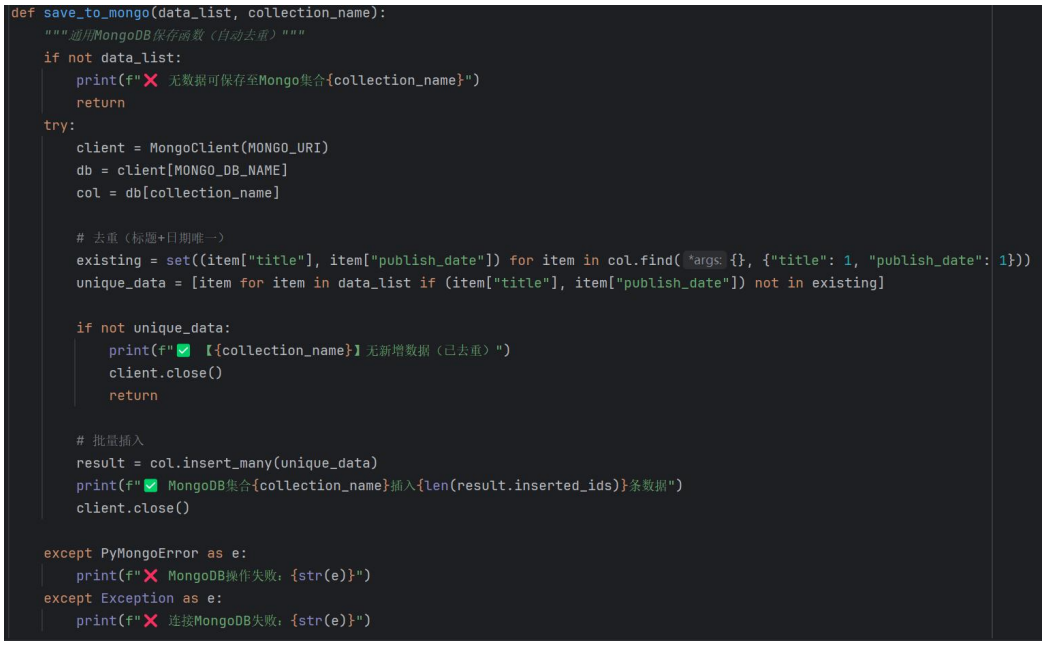
except requests.exceptions.RequestException as e 专门捕获请求相关异常（如网络失败、超时、HTTP 错误），打印 “请求失败” 提示并返回空列表；except Exception as e 捕获其他未预料的异常（如 XPath 语法错误、字段提取失败），打印 “解析失败” 提示并返回空列表。这种机制让函数遇到错误时不会崩溃，而是优雅地返回空列表并给

出明确提示，便于排查问题。整个函数最终返回 news_list，里面是结构化的新闻数据，供后续的 CSV 或 MongoDB 保存逻辑使用。

```python
def save_to_csv(data_list, csv_path):
    """通用CSV保存函数"""
    if not data_list:
        print(f"❌ 无数据可保存至{csv_path}")
        return
    fields = ["tag", "title", "link", "publish_date"]
    try:
        with open(csv_path, "w", encoding="utf-8-sig", newline="") as f:
            writer = csv.DictWriter(f, fieldnames=fields)
            writer.writeheader()
            writer.writerows(data_list)
        print(f"✅ 已保存至CSV：{csv_path}")
    except Exception as e:
        print(f"❌ CSV保存失败：{str(e)}")
```

图 3-10（存储函数）

把之前抓取解析得到的新闻列表，按规范格式写入 CSV 文件。ata_list 是待保存的新闻数据集合（每个元素是包含 “tag、标题、链接、发布日期” 的字典），csv_path 是目标 CSV 文件的本地保存路径。开头的 if not data_list:会先判断数据列表是否为空，若为空则直接打印 “无数据可保存” 的提示并返回，避免无意义的文件操作，提升代码的合理性。fields 列表，明确 CSV 文件的表头字段为["tag", "title", "link", "publish_date"]，这些字段与 data_list 中每个字典的键严格对应，确保后续写入的表头和数据列能精准对齐。通过 with open(...)语句打开文件：其中"w"表示以写入模式操作文件（若文件已存在则覆盖），encoding="utf-8-sig"是为了解决 Excel 打开 CSV 时的乱码问题（utf-8-sig 会添加 BOM 头，让 Excel 正确识别编码），newline=""则避免写入时出现多余的空行。



```python
def save_to_mongo(data_list, collection_name):
    """通用MongoDB保存函数（自动去重）"""
    if not data_list:
        print(f"❌ 无数据可保存至Mongo集合{collection_name}")
        return
    try:
        client = MongoClient(MONGO_URI)
        db = client[MONGO_DB_NAME]
        col = db[collection_name]

        # 去重（标题+日期唯一）
        existing = set((item["title"], item["publish_date"]) for item in col.find( *args: {}, {"title": 1, "publish_date": 1}))
        unique_data = [item for item in data_list if (item["title"], item["publish_date"]) not in existing]

        if not unique_data:
            print(f"✅ 【{collection_name}】无新增数据（已去重）")
            client.close()
            return

        # 批量插入
        result = col.insert_many(unique_data)
        print(f"✅ MongoDB集合{collection_name}插入{len(result.inserted_ids)}条数据")
        client.close()

    except PyMongoError as e:
        print(f"❌ MongoDB操作失败：{str(e)}")
    except Exception as e:
        print(f"❌ 连接MongoDB失败：{str(e)}")
```

图 3-11（存储函数）

data_list 是待保存的结构化新闻数据集合，collection_name 是 MongoDB 中用于存储该类数据的集合名(比如科艺要闻对应 kyyw_news,实现不同栏目数据的分集合存储)。函数开头先判断 data_list 是否为空，若为空则打印 “无数据可保存” 的提示并直接返回，避免无意义的数据库操作。进入 try 块后，首先通过 MongoClient(MONGO_URI)建立与本地 MongoDB 的连接，接着通过 client[MONGO_DB_NAME]选择指定的数据库，再通过 db[collection_name]选择对应的集合——这是 MongoDB 操作的标准流程，完成“客户端→数据库→集合”的层级定位。通过 col.find({}, {"title": 1, "publish_date": 1})查询集合中已有的数据，但仅提取 title 和 publish_date 两个字段(减少数据传输量)；然后将这两个字段组合成元组，存入 existing 集合(集合的 “不重复” 特性可快速实现去重判断)。随后通过列表推导式遍历 data_list，筛选出 “标题 + 发布日期” 元组不在 existing 中的数据，得到去重后的 unique_data—— 这一步是为了避免多次运行爬虫时，重复插入相同的新闻内容，保证数据库中数据的唯一性。

4. 结果展示

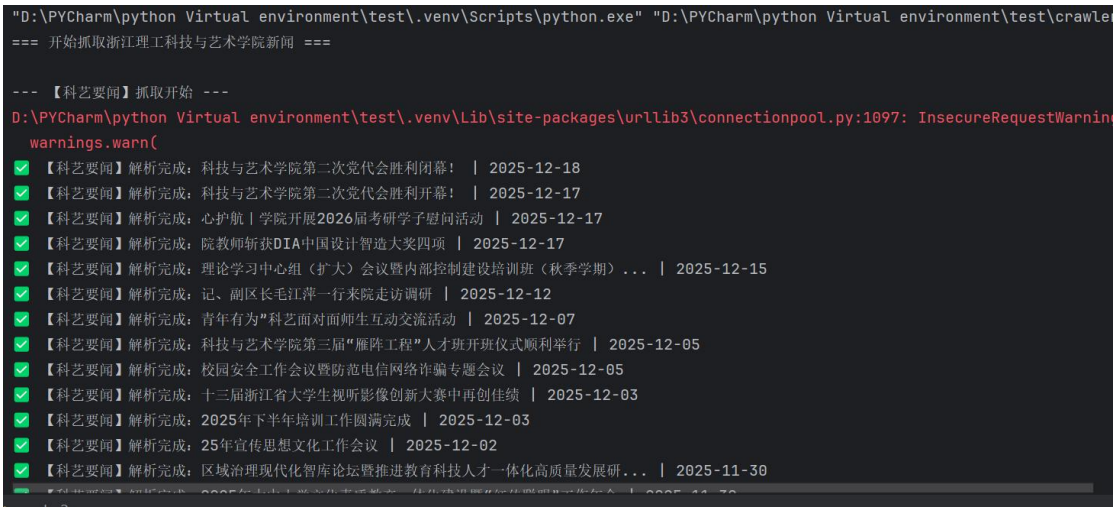


图 3-12（输出结果）


```
_id: ObjectId('694650e9551c3f51f16a54bd')
tag : "浙江理工大学"
title : "科技与艺术学院第二次党代会胜利闭幕！"
link : "https://www.ky.zstu.edu.cn/info/1033/11613.htm"
publish_date : "2025-12-18"

_id: ObjectId('694650e9551c3f51f16a54be')
tag : "浙江理工大学"
title : "科技与艺术学院第二次党代会胜利开幕！"
link : "https://www.ky.zstu.edu.cn/info/1033/11610.htm"
publish_date : "2025-12-17"

_id: ObjectId('694650e9551c3f51f16a54bf')
tag : "研途相伴，暖"
title : "心护航丨学院开展2026届考研学子慰问活动"
link : "https://www.ky.zstu.edu.cn/info/1033/11606.htm"
publish_date : "2025-12-17"

_id: ObjectId('694650e9551c3f51f16a54c0')
tag : "开创先河！我"
title : "院教师斩获DIA中国设计智造大奖四项"
link : "https://www.ky.zstu.edu.cn/info/1033/11604.htm"
publish_date : "2025-12-17"

_id: ObjectId('694650e9551c3f51f16a54c1')
tag : "学院召开党委"
title : "理论学习中心组（扩大）会议暨内部控制建设培训班（秋季学期）..."
link : "https://www.ky.zstu.edu.cn/info/1033/11592.htm"
publish_date : "2025-12-15"

_id: ObjectId('694650e9551c3f51f16a54c2')
tag : "上虞区委副书"
title : "记、副区长毛江萍一行来院走访调研"
link : "https://www.ky.zstu.edu.cn/info/1033/11569.htm"
publish_date : "2025-12-12"
```


图 3-13（输出结果）

任务二：

一、Request 请求的常见状态码及含义

状态码	类别	含义
200	成功类	请求成功，服务器正常返回资源
301	重定向	请求资源已永久迁移到新 URL，后续请求应使用新地址
302	重定向	请求资源临时迁移到新 URL，后续请求仍可使用原地址
400	客户端错误	请求格式错误（如参数不合法），服务器无法识别
401	客户端错误	请求未授权，需携带身份凭证（如 Token、Cookie）
403	客户端错误	服务器拒绝请求（如权限不足）
404	客户端错误	请求资源不存在
500	服务器错误	服务器内部发生未知错误
502	服务器错误	网关/代理服务器收到上游服务器的无效响应

二、Request 请求的标准 HTTP 方法

方法	作用
GET	获取资源
POST	提交资源（创建/修改）
PUT	替换/更新资源
DELETE	删除资源
HEAD	获取响应头（无响应体）
OPTIONS	查看服务器支持的 HTTP 方法
PATCH	部分更新资源

三、URL 输入到页面渲染的完整流程

1、URL 解析：拆分 URL 为协议（如 HTTP）、域名、路径等——无特定协议

2、DNS 解析：将域名转换为 IP 地址——用到 DNS 协议

3、建立 TCP 连接：与目标 IP 的对应端口（HTTP 默认 80、HTTPS 默认 443）完成三次握手——用到 TCP 协议

4、发送 HTTP/HTTPS 请求：向服务器发送请求（含请求头、请求体）——用到 HTTP/HTTPS 协议

5、服务器处理请求并返回响应：服务器处理请求后返回响应（含状态码、响应头、响应体）——用到 HTTP/HTTPS 协议

6、关闭 TCP 连接：完成数据传输后，通过四次挥手关闭连接——用到 TCP 协议

7、浏览器解析响应：解析 HTML（构建 DOM 树）、CSS（构建 CSSOM 树）、执行 JS——无特定协议

8、页面渲染：合并 DOM 树与 CSSOM 树生成渲染树，最终绘制页面——无特定协议

四、常用请求头（RequestHeaders）及其作用

请求头字段	作用
User-Agent	标识请求的客户端类型（如浏览器、爬虫），服务器据此返回适配内容
Accept	告知服务器客户端可接收的资源格式（如 text/html、application/json）
Accept-Encoding	告知服务器客户端支持的压缩格式（如 gzip），减少传输体积
Cookie	携带客户端的身份凭证/状态信息，实现会话保持
Host	标识目标服务器的域名（多域名部署时，服务器据此区分站点）
Referer	标识请求的来源页面，服务器可用于防盗链、统计来源

五、HTTP 与 HTTPS 的主要区别

对比维度	HTTP	HTTPS

协议基础	基于 TCP 裸传输	基于 HTTP+SSL/TLS 加密层（安全传输）
默认端口	80	443
安全性	明文传输，易被窃听/篡改	加密传输，可验证身份、防篡改
证书要求	无需证书	需 CA 颁发的 SSL 证书
性能	传输速度快（无加密开销）	传输速度略慢（需加密/解密开销）

六、Python 中 urllib、urllib3、requests 的差异对比

对比维度	urllib	urllib3	requests
所属	标准库	第三方库	第三方库
请求方式	需构造 Request 对象，API 繁琐	支持连接池，API 较简洁	面向用户的简洁 API
会话保持	不支持（需手动管理 Cookie）	支持	支持（Session 对象）
功能支持	基础 HTTP 请求	支持重试、连接池	自动处理编码、Cookie 等

七、基础爬虫常用第三方库及作用

requests：发送 HTTP/HTTPS 请求（获取页面、接口数据）

BeautifulSoup4：解析 HTML/XML 文档，提取目标数据（如标签内容、属性）

lxml：作为 BeautifulSoup 的解析器（比默认解析器更快、更强大）

pandas：将抓取的数据整理为表格（如 CSV、Excel），方便存储/分析

八、Cookie 与 Session 的区别

对比维度	Cookie	Session
存储位置	客户端（浏览器/本地文件）	服务器端（内存/数据库）
生命周期	可设置过期时间（长期/会话级）	通常随会话结束（如浏览器关闭），或服务器主动销毁

安全性	明文存储	仅存储 SessionID 在 Cookie，数据存服务器
存储容量	有限制（单条约 4KB，单个域名通常最多 20-50 条）	无明确限制
实际存储的常见内容	登录状态标识、客户端配置、临时业务数据、安全相关令牌、第三方标识	用户核心身份信息、权限与认证相关数据、业务流程临时数据、会话状态控制数据、复杂业务对象

九、BeautifulSoup 解析 HTML 的常用方法及功能

find()：查找符合条件的第一个标签

find_all()：查找符合条件的所有标签，返回列表

select()：通过 CSS 选择器查找标签

get_text()：提取标签内的文本内容

attrs：获取标签的属性值

十、提升爬虫抓取速度的常用手段

多线程/多进程：同时发起多个请求（利用 CPU 多核资源）

异步请求：用 aiohttp 等库实现非阻塞请求（减少等待时间）

分布式爬虫：多台机器同时抓取（适合大规模数据）

缓存请求结果：重复请求的资源直接用缓存（减少请求次数）

使用代理池：避免单 IP 被限速，同时并行请求

十一、常见反爬虫策略及应对思路

反爬虫策略	应对思路
User-Agent 检测	轮换不同的 User-Agent
IP 访问频率限制	使用代理池切换 IP，或添加随机请求间隔
验证码拦截	借助 OCR 工具识别，或人工打码
防盗链（Referer 检测）	伪造 Referer 字段

动态渲染页面（JS 加载）	使用 Selenium/Playwright 模拟浏览器渲染，执行 JS 后再抓取

四、心得体会

这次做这个学院官网爬虫的项目，说实在的，一开始我真没觉得有多复杂 —— 不就是发个请求、扒点新闻、存起来嘛？结果上手才发现，全是 “看起来简单、做起来踩坑” 的细节。

最开始写请求的时候，我直接用 requests.get 扔了个 URL 过去，结果返回的页面里标题全是 “锟斤拷” 这种乱码，盯着屏幕愣了好半天。一开始以为是网站编码的问题，试了 utf-8、gbk 各种编码手动转，都没用；后来查了半天才知道有个 response.apparent_encoding，能自动识别页面真实编码，加上这行代码的瞬间，乱码突然变成了正常的中文，那种 “终于搞定了” 的踏实感，现在还记得。

然后是去重的事儿。第一次跑爬虫，MongoDB 里一下插了 20 多条新闻，第二次跑的时候没多想直接运行，结果数据库里同一篇新闻存了两遍。当时我还纳闷 “怎么数据重复了”，后来才反应过来：爬虫每次运行都会抓最新的列表，但旧新闻其实已经存过了。最开始想只用标题去重，结果发现学院有两篇 “校园安全会议” 的新闻，标题一样但日期不同，这方法不行；最后才想到把 “标题 + 发布日期” 绑在一起当唯一标识，这才把重复的问题彻底解决 —— 现在再看那段去重的代码，都是试错试出来的经验。

还有反爬那事儿，也挺闹心的。一开始我没加 time.sleep，写完代码连着跑了三次，结果第四次直接返回 403 错误。当时还以为是代码写错了，反复检查请求头、URL，折腾了快一小时，才反应过来是访问太频繁被网站拦了。加上 1 秒延迟之后再跑，就顺顺当当的了 —— 这才明白，爬虫不是 “越快越好”，得跟着网站的节奏来，不然 “爬着爬着就被拉黑” 真不是开玩笑。

最让我有感触的是代码模块化。最开始我把请求、解析、保存全堆在 main 函数里，写科艺要闻的时候还行，后来要加校园经纬栏目，改代码改得头晕脑胀 —— 改一处逻辑，两处都要跟着调。后来咬牙把代码拆成了 fetch_news_data、save_to_csv、save_to_mongo 这几个函数，后面加新栏目时，只需要加个配置、调用一下 fetch 函数就行，效率一下高了不少。这才真懂了 “单一职责” 不是书本里的空话，是真能少走弯路的实用方法。

现在再看这个爬虫，其实功能不算复杂，但从头到尾踩的每一个坑，都让我对 “爬虫” 的理解变了 —— 它不是 “薅数据” 的工具，是得考虑请求合规、数据准确、存储合理的活儿。比如存数据的时候，CSV 适合我自己打开看，MongoDB 适合存多了之后查；比如解析页面得盯着网站的结构来，万一网站改了 class 名，代码就得跟着调。

折腾这一趟下来，最大的收获不是写了个能跑的爬虫，是学会了 “遇到问题别慌” —— 乱码了就查编码，重复了就想去重规则，被拦了就调访问节奏，一点点试、一点点改，比盯着代码干想有用多了。

五、核心代码

```
import requests

from lxml import etree

import csv

import time

from pymongo import MongoClient

from pymongo.errors import PyMongoError


# -------------------------- 通用配置项 --------------------------

HEADERS = {

    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)

Chrome/120.0.0.0 Safari/537.36",

    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",

    "Accept-Language": "zh-CN,zh;q=0.9"

}

MONGO_URI = "mongodb://localhost:27017/"

MONGO_DB_NAME = "school_news_db"  # 共用数据库

DELAY = 1  # 防反爬延迟


# -------------------------- 科艺要闻配置 --------------------------

KYYW_TARGET_URL = "https://www.ky.zstu.edu.cn/kyyw.htm"
```

```
KYYW_CSV_PATH = "浙江理工科技与艺术学院科艺要闻.csv"
KYYW_MONGO_COL = "kyyw_news"

# -------------------------- 校园经纬配置（新增） --------------------------
XYJW_TARGET_URL = "https://www.ky.zstu.edu.cn/xyjw.htm"  # 对应你提供的页面 URL
XYJW_CSV_PATH = "浙江理工科技与艺术学院校园经纬.csv"
XYJW_MONGO_COL = "xyjw_news"



# -------------------------- 通用数据抓取函数 --------------------------
def fetch_news_data(target_url, tag_name):
    """通用新闻抓取解析函数（适配科艺要闻/校园经纬）"""
    try:
        time.sleep(DELAY)
        # 发送请求
        response = requests.get(
            url=target_url,
            headers=HEADERS,
            timeout=15,
            verify=False
        )
        response.raise_for_status()
        response.encoding = response.apparent_encoding
        html = etree.HTML(response.text)

        # 定位新闻条目（页面结构与原科艺要闻一致）
        news_items = html.xpath('//div[@class="wlistee"]/ul/li')
        if not news_items:
            print(f"❌ 未找到【{tag_name}】条目，请检查网页结构")
            return []
```

```
        news_list = []
        for item in news_items:
            # 提取标签+标题（如【校园经纬】xxx）
            a_text = item.xpath('.//a/text()')[0].strip() if item.xpath('.//a/text()') else ""
            tag = a_text[:6]  # 标签固定6个字符（如【校园经纬】）
            title = a_text[6:].strip()

            # 提取并拼接链接
            link = item.xpath('.//a/@href')[0].strip() if item.xpath('.//a/@href') else ""
            if link and not link.startswith(("http://", "https://")):
                link = f"https://www.ky.zstu.edu.cn/{link}"

            # 提取发布日期
            publish_date = item.xpath('.//span/text()')[0].strip() if item.xpath('.//span/text()') else
""

            # 组装数据
            news_list.append({
                "tag": tag,
                "title": title,
                "link": link,
                "publish_date": publish_date
            })
            print(f"☑ 【{tag_name}】解析完成：{title} | {publish_date}")

        return news_list

    except requests.exceptions.RequestException as e:
        print(f"❌ 【{tag_name}】请求失败：{str(e)}")
```

```
        return []
    except Exception as e:
        print(f"❌【{tag_name}】解析失败：{str(e)}")
        return []


# -------------------------- 通用数据保存函数 --------------------------
def save_to_csv(data_list, csv_path):
    """通用 CSV 保存函数"""
    if not data_list:
        print(f"❌无数据可保存至{csv_path}")
        return
    fields = ["tag", "title", "link", "publish_date"]
    try:
        with open(csv_path, "w", encoding="utf-8-sig", newline="") as f:
            writer = csv.DictWriter(f, fieldnames=fields)
            writer.writeheader()
            writer.writerows(data_list)
        print(f"☑已保存至 CSV：{csv_path}")
    except Exception as e:
        print(f"❌CSV 保存失败：{str(e)}")


def save_to_mongo(data_list, collection_name):
    """通用 MongoDB 保存函数（自动去重）"""
    if not data_list:
        print(f"❌无数据可保存至 Mongo 集合{collection_name}")
        return
    try:
        client = MongoClient(MONGO_URI)
```

```
        db = client[MONGO_DB_NAME]
        col = db[collection_name]

        # 去重（标题+日期唯一）
        existing = set((item["title"], item["publish_date"]) for item in col.find({}, {"title":1,
"publish_date":1}))
        unique_data = [item for item in data_list if (item["title"], item["publish_date"]) not in existing]

        if not unique_data:
            print(f"☑ 【{collection_name}】无新增数据（已去重）")
            client.close()
            return

        # 批量插入
        result = col.insert_many(unique_data)
        print(f"☑ MongoDB 集合{collection_name}插入{len(result.inserted_ids)}条数据")
        client.close()

    except PyMongoError as e:
        print(f"❌ MongoDB 操作失败：{str(e)}")
    except Exception as e:
        print(f"❌ 连接 MongoDB 失败：{str(e)}")


# -------------------------- 主程序（同时爬取两个页面） --------------------------
def main():
    print("=== 开始抓取浙江理工科技与艺术学院新闻 ===")

    # 1. 爬取科艺要闻
    print("\n--- 【科艺要闻】抓取开始 ---")
```

```
    kyyw_data = fetch_news_data(KYYW_TARGET_URL, "科艺要闻")
    if kyyw_data:
        save_to_csv(kyyw_data, KYYW_CSV_PATH)
        save_to_mongo(kyyw_data, KYYW_MONGO_COL)
    else:
        print("--- 【科艺要闻】无数据 ---")

    # 2. 爬取校园经纬（新增）
    print("\n--- 【校园经纬】抓取开始 ---")
    xyjw_data = fetch_news_data(XYJW_TARGET_URL, "校园经纬")
    if xyjw_data:
        save_to_csv(xyjw_data, XYJW_CSV_PATH)
        save_to_mongo(xyjw_data, XYJW_MONGO_COL)
    else:
        print("--- 【校园经纬】无数据 ---")

    print("\n=== 所有抓取任务完成 ===")


if __name__ == "__main__":
    main()
```

指导老师评价：

项目评价	优	良	中	一般	较差
课堂纪律：按时上课，无迟到早退					
完成态度：认真完成大作业设计，勤奋好学，积极思考					
项目完成度：按照要求完成项目设计					
课程质量：代码正确，按时完成，界面美观					
课程总结：思路清晰，调理清楚，全面总结了心得体会和不足					

总评成绩	**签名：** **时间：**